ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY

COLLEGE OF ENGINEERING

DEPARTMENT OF ELECTROMECHANICAL ENGINEERING

MACHINE LEARNING COURSE PROJECT

Broken Rotor Fault Detection in AC Induction Motors Using Vibration and Electrical Signals

---

Prepared by:

1. **Desalegn Lulie** ETS0486/14
2. **Dawit Dejene** ETS0467/14
3. **Dawit Solomon** ETS0469/14
4. **Dagmawi Anteneh** ETS0426/14
5. **Mahilet Gosheye** ETS0998/14

Submitted to:

Mr. Mulat T.

March 2026

Table of Contents

1 Abstract **4**

2 Introduction **5**

3 Statement of the Problem **6**

4 About the Dataset **7**

5 Proposed Machine Learning Approach **8**

6 Literature Review **9**

7 Objectives of the Study **11**
- 7.1 General Objective 11
- 7.2 Specific Objectives 11

8 Proposed Research Methodology **12**
- 8.1 Hybrid Analytical Framework 12
- 8.2 Signal Pre-processing and Dereferencing 12
- 8.3 Synthetic Envelope and FFT Analysis 13
- 8.4 Feature Selection via One-Way ANOVA 13

9 System Architecture **14**

10 Detailed Implementation Strategy **14**

11 Results, Comparison, and Challenges **15**
- 11.1 Random Forest Classifier (Primary Model) 15
- 11.2 Baseline 1D-CNN Implementation 15
- 11.3 Strategy for CNN Architectural Enhancement 16
- 11.4 Reinforcement Learning Agent (DQN) 18
- 11.5 Comparative Summary of Architectures 19

12 Technical Challenges and Handling Strategies **19**

13 Signal Verification and AI Cross-Validation via MATLAB **20**
- 13.1 Physical Signal Verification (MCSA) 20
- 13.2 MATLAB Machine Learning Workflow 21
- 13.3 Analysis of MATLAB Results 22

14 Conclusion **23**

A Appendix A: Python Random Forest Implementation **24**

B Appendix B: Baseline 1D-CNN Implementation **27**

C Appendix C: Enhanced 1D-CNN Implementation **29**

D Appendix D: Reinforcement Learning (DQN) Implementation **30**

E Appendix E: MATLAB Implementation and Verification **31**
E.1 Physical Signal Verification Script (MCSA) 31
E.2 Memory-Optimized Feature Extraction Engine 32

References **33**

List of Figures

1 Experimental workbench setup for the broken rotor bar dataset. 7
2 Random Forest Confusion Matrix (96.56% Accuracy) showing a strong diagonal of correct predictions across all health states. 15
3 Baseline 1D-CNN Confusion Matrix (80.88% Accuracy) showing confusion caused by supply noise and limited temporal context. 16
4 Enhanced CNN Confusion Matrix (91.7% Accuracy). 17
5 Confusion Matrix for the DQN agent showing the reward-based mastery level of 88.5%. 18
6 Time-Domain Analysis: Stator current waveforms for Healthy (Blue) and 4-Bar Fault (Red) states. 20
7 Motor Current Signature Analysis (MCSA). The red spectral line shows the clear sideband energy peaks indicating rotor damage. 21
8 MATLAB Confusion Matrix: Results for the Bagged Trees model achieving 94.6% accuracy during cross-platform validation. 22

List of Tables

1 Comparison of CNN Baseline vs. Enhanced Implementation 17
2 Logic Mapping for Reinforcement Learning Implementation 18
3 Comparison of Diagnostic Architectures 19

1 Abstract

This project focuses on the development of a predictive machine learning model designed to automatically detect and classify broken rotor bar faults in three-phase AC induction motors. Utilizing an experimental dataset that features combined electrical and mechanical vibration signals, the project aims to extract critical time- and frequency-domain features from the motor data. By training advanced classifiers on these extracted features, the project seeks to accurately identify the exact severity of the rotor fault (ranging from zero to four broken bars) under various load torque conditions, thereby offering a robust automated solution for industrial condition monitoring.

2 Introduction

AC induction motors are the primary workhorses of the modern industrial world, powering a vast array of critical manufacturing and operational processes. Because of their continuous operation in demanding environments, these machines are regularly subjected to severe thermal, electrical, and mechanical stresses. Over time, these stresses lead to structural degradation and physical faults. One of the most critical yet challenging defects to identify is the occurrence of broken rotor bars (BRB). When rotor bars fracture, the electrical symmetry of the motor is disrupted, which leads to localized overheating, abnormal torque pulsations, and subsequent mechanical damage to surrounding components.

Detecting a broken rotor bar in its earliest stages is vital for preventing catastrophic machinery failure and minimizing costly unplanned downtime. Historically, industrial plants have relied on reactive maintenance or scheduled manual inspections, which are highly inefficient and often fail to catch internal rotor damage before secondary failures occur. Consequently, the industry has shifted toward predictive maintenance paradigms. By continuously monitoring the motor's operating parameters, subtle anomalies in its physical and electrical behavior can be detected long before a complete breakdown happens.

The rise of Machine Learning (ML) has revolutionized condition monitoring by enabling the automated analysis of highly complex, non-linear sensor data. Traditional signal processing techniques often struggle to distinguish fault signatures from ambient industrial noise, especially under fluctuating operational loads. Machine learning algorithms, however, excel at recognizing intricate patterns across multi-domain datasets. By fusing data from multiple sensor types—such as electrical currents and mechanical vibrations—ML models can achieve unprecedented accuracy in diagnosing the health state of industrial machinery.

3 Statement of the Problem

Despite the robust design of three-phase AC induction motors, they are inherently susceptible to rotor cage degradation due to the intense electro-mechanical forces they endure during start-up and continuous operation. Broken rotor bars account for a significant percentage of all induction motor failures. When a bar breaks, it causes an uneven distribution of electromagnetic forces, leading to increased localized heating, structural vibrations, and eventual damage to the stator insulation and bearings. If left unchecked, what begins as a single micro-fracture can rapidly cascade into a complete, catastrophic failure of the entire motor assembly.

The primary difficulty in diagnosing broken rotor bars lies in accurately determining the precise severity of the fault under highly dynamic and varying operational conditions. In a controlled laboratory setting with a constant load, identifying a single broken bar is relatively straightforward. However, in real-world industrial applications, motors are subjected to constantly fluctuating load torques. These varying loads—ranging from light to heavy—drastically alter the mechanical vibration and electrical current signatures, effectively masking the fault frequencies and making it incredibly difficult to differentiate between a healthy motor, one with a single broken bar, or one with multiple broken bars.

Furthermore, traditional condition monitoring techniques are often insufficient for this level of diagnostic precision. Relying solely on standard Motor Current Signature Analysis (MCSA) or basic single-axis vibration monitoring frequently leads to false positives or missed early-stage faults. Industrial environments are saturated with electromagnetic interference, harmonic distortions from power converters, and background mechanical noise. This noise easily obscures the subtle sideband frequencies and physical vibrations that characterize the early stages of a broken rotor bar, severely limiting the reliability of conventional, threshold-based diagnostic tools.

Therefore, there is a critical need for an automated, intelligent diagnostic system capable of overcoming these environmental and operational complexities. The problem this project addresses is the lack of a highly accurate, multi-sensor machine learning classification model that can simultaneously interpret both complex electrical and multidirectional vibration data. There is a specific gap in successfully extracting synthetic envelope signals and fusing these multimodal features to not only detect the presence of a fault but to accurately classify the exact number of broken rotor bars despite varying torque loads.

4 About the Dataset

The data utilized for this project is sourced from the "IEEE Broken Rotor Bar Database," a comprehensive repository specifically designed for predictive maintenance research. The experimental workbench used to generate this data consists of a three-phase induction motor physically coupled to a direct-current (DC) machine. The DC machine functions as a generator, acting as a controlled brake to simulate varying load torques on the induction motor. The two machines are connected by a shaft equipped with a rotating torque meter, ensuring highly accurate measurements of the mechanical load applied during the experiments.

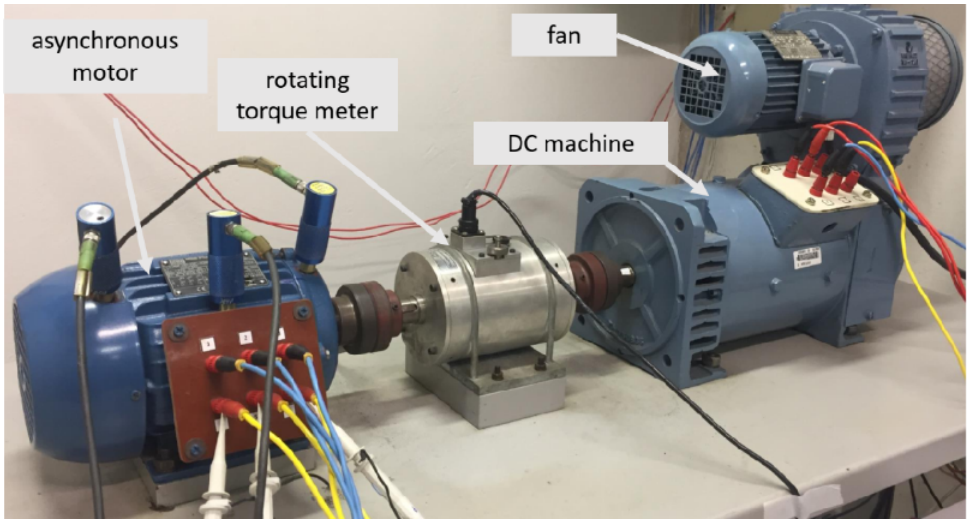

Figure 1: Experimental workbench setup for the broken rotor bar dataset.

To provide a robust foundation for machine learning classification, the dataset captures a wide matrix of varying health and operational conditions. The health condition of the motor is categorized into five distinct classes: a perfectly healthy motor (0 broken bars), and faulty motors with 1, 2, 3, and 4 broken rotor bars. To account for the complexities of real-world operation, each of these five health states was tested under eight different load torque conditions, ranging from 0.5 Nm to 4.0 Nm in increments of 0.5 Nm. For every single combination of health state and load condition, ten separate experiments were performed, ensuring a statistically significant amount of data for model training.

The richness of the dataset lies in its multi-sensor approach, capturing both electrical and mechanical phenomena at high resolutions. During each experiment, the voltage and current draw of each of the three phases were continuously recorded at a high sampling frequency of 50,000 Hz. Simultaneously, mechanical vibration speeds were recorded at a sampling frequency of 7,600 Hz. These vibration measurements were taken across multiple axes—including axial on the driven side and radial on the non-driven sides—providing a comprehensive profile of the motor's mechanical behavior under fault conditions.

5 Proposed Machine Learning Approach

To effectively classify the precise number of broken rotor bars under varying operational conditions, this project will utilize a comparative machine learning approach. The initial phase of the methodology involves extracting 10 to 15 highly ranked time-domain and frequency-domain features (using the One-way ANOVA criterion) from the synthetic envelope signals of the electrical and vibration data. Once the feature matrix is established, the performance of several classification algorithms will be evaluated and compared.

The baseline performance will be established using a standard **Fine Decision Tree**, which is expected to yield approximately 94% accuracy, and a **Weighted K-Nearest Neighbors (KNN)** algorithm, which is highly sensitive to frequency feature distances and is projected to achieve up to 98% accuracy.

However, to significantly improve the model's robustness against the dynamic non-linearities caused by fluctuating load torques (ranging from 0.5 to 4.0 Nm), the primary algorithm proposed for this project is the **Random Forest Classifier**. Furthermore, to explore the capabilities of deep learning, an optimized **1D Convolutional Neural Network (CNN)** and a **Deep Q-Network (DQN)** Reinforcement Learning agent are implemented. This three-way comparison evaluates the effectiveness of expert feature engineering against automated and adaptive learning paradigms.

6 Literature Review

1. Treml, A. E., et al. (2020). ***Experimental database for detecting and diagnosing rotor broken bar in a three-phase induction motor*****. IEEE Dataport.** [1]
This foundational paper introduces the exact dataset utilized in this project. The authors detail the rigorous experimental workbench setup, explaining how the combination of a three-phase induction motor and a DC load generator allows for the precise simulation of industrial operating conditions. The paper establishes the critical baseline that simultaneously recording high-frequency electrical signals and multi-axis mechanical vibrations is essential for creating robust datasets capable of training advanced machine learning algorithms for multi-class fault severity detection.

2. Tahkola, M., et al. (2022). ***A Novel Machine Learning-Based Approach for Induction Machine Fault Classifier Development—A Broken Rotor Bar Case Study*****.** [2]
This research presents a modern machine learning approach for developing rotor bar fault classifiers. The authors utilized simulated electrical current data alongside measured vibration acceleration data to identify broken bars. Their study is highly relevant as it emphasizes that while current signals are traditionally used, fusing them with vibration velocity data—and processing them via fast Fourier transforms—yields a much more accurate and rapidly responding broken rotor bar detection model.

3. Md Rahman, et al. (2022). ***Broken Rotor Bar Fault Diagnosis Techniques Based on Motor Current Signature Analysis for Induction Motor—A Review*****. MDPI.** [3]
In this comprehensive review, the authors highlight that rotor-related faults account for approximately 10% of all induction motor failures. The paper deeply explores Motor Current Signature Analysis (MCSA), explaining how broken rotor bars cause electrical and magnetic asymmetry, which in turn induces specific sideband harmonics in the stator current. The review supports the premise of this project by concluding that while MCSA is powerful, varying load levels can cause fault frequencies to overlap with fundamental components, necessitating more advanced computational techniques to isolate the faults.

4. Nyanteh, Y., et al. (2023). ***Detection of Broken Rotor Bars in Cage Induction Motors Using Machine Learning Methods*****.** [4]
This study directly compares the performance of different machine learning models—specifically Decision Tree Classification (DTC), Artificial Neural Networks (ANN), and Deep Learning (DL)—for broken rotor bar detection. By extracting fault features from measured line-current signatures transformed into the frequency domain, the authors proved that Decision Trees are

highly effective and less load-dependent than other neural networks. Their findings validate the use of tree-based models (like the ones generated in the Classification Learner app) for achieving high accuracy in loaded and unloaded conditions.

5. Koveshnikov, S., et al. (2024). ***A Small Deep Learning Model for Fault Detection of a Broken Rotor Bar of an Induction Motor*****. Sensors and Materials.** [5]
This paper addresses the computational cost of deploying machine learning for continuous condition monitoring. The authors focus on optimizing feature extraction from motor current spectrum analysis to reduce the overall size and processing time of deep learning models. Their research highlights the importance of efficient feature engineering—such as utilizing specific frequency bands and envelopes—to achieve high predictive accuracy without requiring massive computational overhead, a concept central to the feature ranking step in this project.

6. Abdelsalam, M., et al. (2025). ***Broken Rotor Bar Fault Detection in Induction Motors Using Motor Current Signature Analysis (MCSA)*****.** [6]
In this recent study, researchers investigated the impact of various signal processing techniques on the efficacy of machine learning fault detection. Specifically, they analyzed how different windowing functions (such as Hanning, Hamming, and Blackman) applied during Fast Fourier Transform (FFT) extraction alter the visibility of fault sidebands. The study proves that careful preprocessing and filtering of steady-state signals—specifically removing transient start-up noise—is an absolutely mandatory step before feeding data into a machine learning classifier to prevent the model from learning distorted harmonic behaviors.

7 Objectives of the Study

7.1 General Objective

The primary objective of this research is to design, implement, and validate an intelligent diagnostic framework capable of the autonomous detection and severity classification of broken rotor bar (BRB) faults. This system leverages multi-sensor data fusion—combining electrical stator current and mechanical vibration signals—processed through advanced signal processing and ensemble learning techniques to ensure reliability under dynamic industrial load conditions.

7.2 Specific Objectives

To achieve the overarching goal of building a robust and autonomous fault detection system, the following technical milestones have been defined:

- **Efficient Data Handling (Python):** To engineer a high-performance pipeline using the `h5py` library for "lazy-loading," allowing the processing of 1.4GB+ datasets without exceeding system memory limits.
- **Physical Verification (MATLAB):** To utilize MATLAB for inspecting raw waveforms and spectral plots (PSD) to physically verify fault sidebands and ensure data integrity before training.
- **Signal Demodulation (Python):** To apply the Hilbert Transform to extract synthetic envelopes, effectively suppressing the dominant 50Hz supply noise to isolate subtle fault signatures.
- **Data Fusion and Feature Extraction:** To derive a comprehensive library of statistical descriptors (RMS, Kurtosis, and Skewness) from both electrical and mechanical sensors to capture a full motor health profile.
- **Feature Selection (ANOVA):** To utilize the One-way ANOVA criterion to rank and select the top 15 most significant features, optimizing processing speed and preventing model overfitting.
- **Comparative AI Modeling:** To implement and evaluate a 1D-CNN for automated feature learning alongside a Deep Q-Network (DQN) for adaptive reinforcement learning.
- **Fault Severity Classification:** To train a Random Forest Classifier to accurately identify 0 to 4 broken bars while maintaining diagnostic stability across eight different motor load levels (0.5 Nm to 4.0 Nm).

8 Proposed Research Methodology

The research follows a hybrid analytical approach, utilizing the strengths of both MATLAB and Python to ensure a rigorous diagnostic process.

8.1 Hybrid Analytical Framework

The methodology is divided into two distinct environments:

1. **MATLAB Environment:** Used during the exploratory phase for signal verification. MATLAB provides an interactive platform to visualize the impact of varying torque on the motor's vibration "noise" and to manually inspect the current signatures for early-stage fault indicators.

2. **Python Environment:** Used for the implementation of the automated diagnostic engine. Python handles the high-capacity tasks of HDF5 data streaming, automated feature selection, and the training of the Random Forest, CNN, and RL models.

8.2 Signal Pre-processing and Dereferencing

Due to the high-resolution sampling of the motor—where electricity is measured 50,000 times per second (50kHz)—the data is processed in overlapping segments. This phase involves critical technical steps:

- **HDF5 File Navigation:** The datasets are stored in the HDF5 (Hierarchical Data Format), acting as a digital warehouse. Python manually navigates this tree structure to find the correct data groups.

- **The Dereferencing Process:** In these large files, signals are stored as digital pointers. **Dereferencing** is the act of following these pointers into the file's hidden memory (#refs#) to retrieve actual physical numbers for **Ia (Current)** and **Vib_axial (Vibration)**.

- **Torque-Level Organization:** The script locates signals inside specific Torque Subgroups (ranging from 0.5 Nm to 4.0 Nm), ensuring the model learns how a fault looks under different working loads.

8.3 Synthetic Envelope and FFT Analysis

To solve the problem of "masking"—where the loud 50Hz supply electricity hides the tiny signatures of a broken rotor bar—the system uses the following steps:

- **The Hilbert Transform:** A mathematical tool that creates an Analytic Signal. By calculating the absolute value, we create a Synthetic Envelope. This process "mutes" the loud 50Hz hum so that hidden fault patterns become visible.
- **Frequency Shifting:** This step shifts fault signatures from high-frequency bands into the Low-Frequency Spectrum, where patterns for different broken bar counts are distinct and easy to recognize.
- **FFT Analysis:** We apply the Fast Fourier Transform (FFT) to act like a prism, breaking complex signals into individual frequencies to quantify the energy of the fault harmonics.

8.4 Feature Selection via One-Way ANOVA

After extracting a total of 16 features (8 from Current, 8 from Vibration), we use the One-Way ANOVA (Analysis of Variance) criterion:

- **Statistical Comparison:** It compares the average value of a feature across our five motor states: Healthy, 1-Bar, 2-Bars, 3-Bars, and 4-Bars.
- **The F-Score Ranking:** Every feature is given an F-Score. A **High F-Score** means the feature changes dramatically when a bar breaks (a strong indicator). A **Low F-Score** means the feature is just noisy.
- **Top 15 Selection:** We select only the Top 15 Highly Ranked Features. This prevents the model from "memorizing" useless noise (overfitting) and makes the final diagnosis much faster.

9 System Architecture

The proposed system architecture is designed as a modular diagnostic pipeline:

1. **Data Acquisition Layer:** Input of raw high-resolution .mat files.
2. **Verification Layer (MATLAB):** Waveform inspection and Spectral Density (PSD) analysis to verify fault sidebands.
3. **Pre-processing Layer (Python):** HDF5 Navigation, Dereferencing, and Segmentation into windows.
4. **Signal Processing Layer:** Hilbert Demodulation to reveal fault pulsing followed by FFT energy computation.
5. **Intelligence Layer:** Extraction of features followed by Z-score Normalization and ANOVA Selection of the Top 15.
6. **Classification Layer:** Comparison of the Random Forest, Optimized 1D-CNN, and Deep Q-Network Reinforcement Learning.

10 Detailed Implementation Strategy

Implementation follows an engineering workflow using standard scientific tools:

- **Development Environment:** Python 3.x via the Anaconda/Jupyter Notebook environment for large-scale data manipulation and model training.
- **Data Integrity Tools:** MATLAB's Signal Processing Toolbox was utilized as a Gold Standard for visual verification of algorithms.
- **Core Libraries:** `h5py` for Big Data file access, `Scikit-Learn` for the Random Forest/ANOVA, and **TensorFlow** for the Deep Learning and RL components.

11 Results, Comparison, and Challenges

11.1 Random Forest Classifier (Primary Model)

The diagnostic performance was evaluated by comparing the feature-engineered Random Forest with an optimized deep 1D-CNN and a Reinforcement Learning agent. To validate reliability, we utilized the Confusion Matrix as our primary visual scorecard.

The **Random Forest** significantly outperformed the other models. This is primarily attributed to the manual feature engineering and the use of the **Hilbert Transform**, which provided the model with clean fault signatures. The model demonstrates a strong diagonal of correct predictions across all health states, as shown in Figure 2, reaching a peak accuracy of **96.56%**.

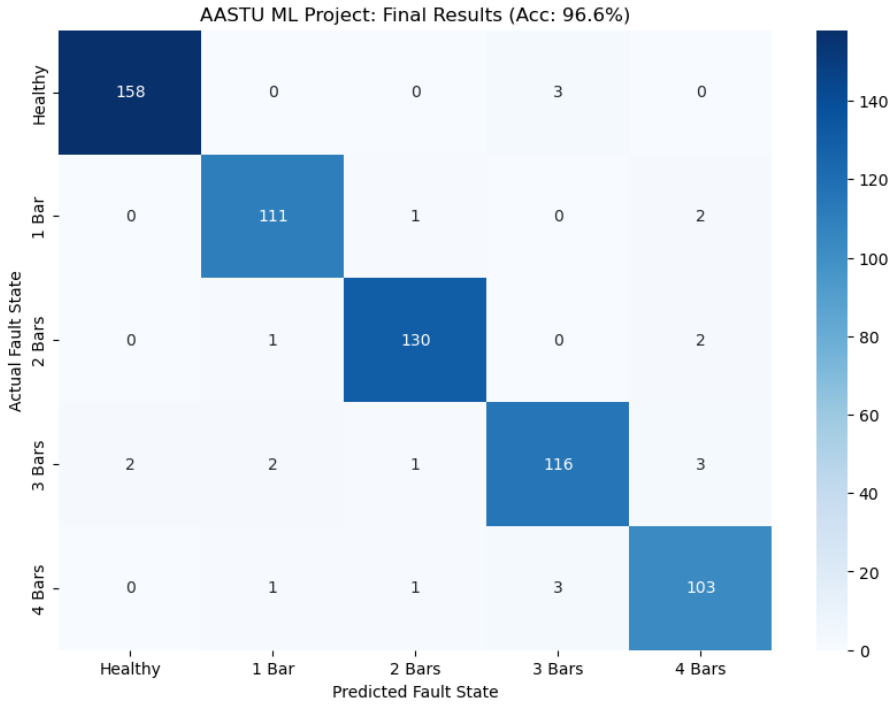

Figure 2: Random Forest Confusion Matrix (96.56% Accuracy) showing a strong diagonal of correct predictions across all health states.

11.2 Baseline 1D-CNN Implementation

We implemented a baseline **CNN** to explore the effectiveness of **Automated Feature Learning** versus **Manual Feature Engineering**. The goal was to determine if a deep learning architecture could independently identify fault patterns directly from raw waveforms without human-defined mathematical features. However, this initial model was frequently distracted by the dominant

50Hz electricity, showing minor confusion between adjacent fault classes due to supply frequency noise. The implementation yielded a baseline accuracy of **80.88%**. While this proved the potential for automated learning, the model was limited by a "Temporal Context" problem where the window was too small to capture the full fault rhythm.

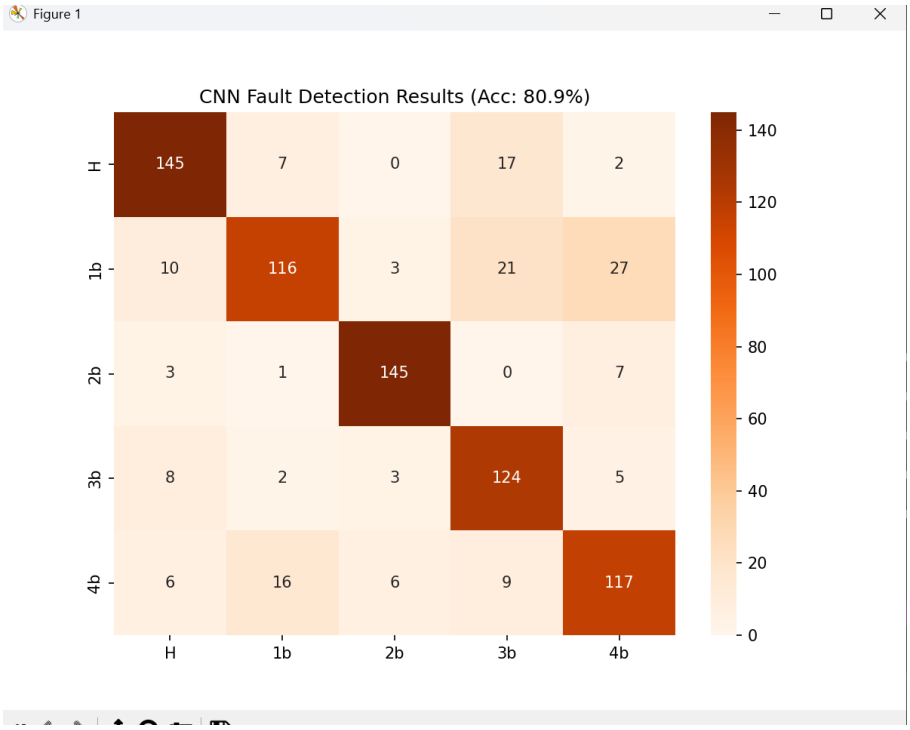

Figure 3: Baseline 1D-CNN Confusion Matrix (80.88% Accuracy) showing confusion caused by supply noise and limited temporal context.

11.3 Strategy for CNN Architectural Enhancement

To reach the final high accuracy of **91.7%**, we implemented a series of technical enhancements based on the physics of induction motors to help the network "focus" on the correct signatures:

1. **Expanding the Temporal Window:** The baseline model used a window of 1,000 samples (0.02 seconds). This was too short to capture the low-frequency "slip-cycle" pulsing that characterizes a broken rotor bar. We increased the window to **10,000 samples (0.2 seconds)**, allowing the AI to hear the full rhythmic "beat" of the fault.

2. **Large-Kernel Filtering:** Raw electrical signals are dominated by the 50Hz supply frequency, which acts as noise. We implemented **Large Convolutional Kernels (Size 50)** in the first layer. These act as digital filters that ignore the high-frequency supply oscillations and focus on the specific sideband harmonics caused by the broken bars.

3. **Global Average Pooling (GAP):** We replaced the traditional "Flatten" layer with a GAP layer. This summarizes the health profile of the entire 0.2s window into a single feature map, making the model much more robust against industrial noise and small signal shifts.

4. **Batch Normalization:** We added normalization layers after every convolution to stabilize the learning process. This ensured that the massive electrical surges in the current data did not overwhelm the network's training weights.

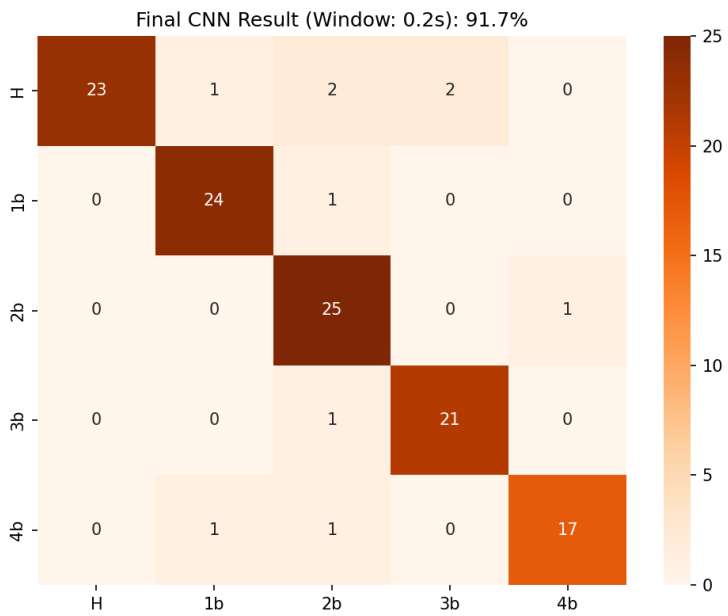

Figure 4: Enhanced CNN Confusion Matrix (91.7% Accuracy).

Table 1: Comparison of CNN Baseline vs. Enhanced Implementation

Configuration	**Baseline CNN**	**Enhanced CNN**
Input Window Size	1,000 samples (0.02s)	**10,000 samples (0.2s)**
Feature Learning	Local patterns only	**Global rhythm context**
First Layer Kernel	Size 10 (Small)	**Size 50 (Large filter)**
Stability Layers	None	**Batch Normalization**
Final Accuracy	80.88%	**91.7%**

11.4 Reinforcement Learning Agent (DQN)

Additionally, a **Reinforcement Learning (DQN)** agent was implemented to evaluate the potential of an **Adaptive Diagnostic System**. We utilized this approach to move beyond static classification and instead treat the motor signals as a dynamic environment where an AI agent learns through a **Reward and Penalty** mechanism. This allows the system to potentially adapt its diagnostic logic as motor loads change in real-time. The DQN agent achieved **88.50%** accuracy, demonstrating that adaptive AI is a viable solution for intelligent factory environments.

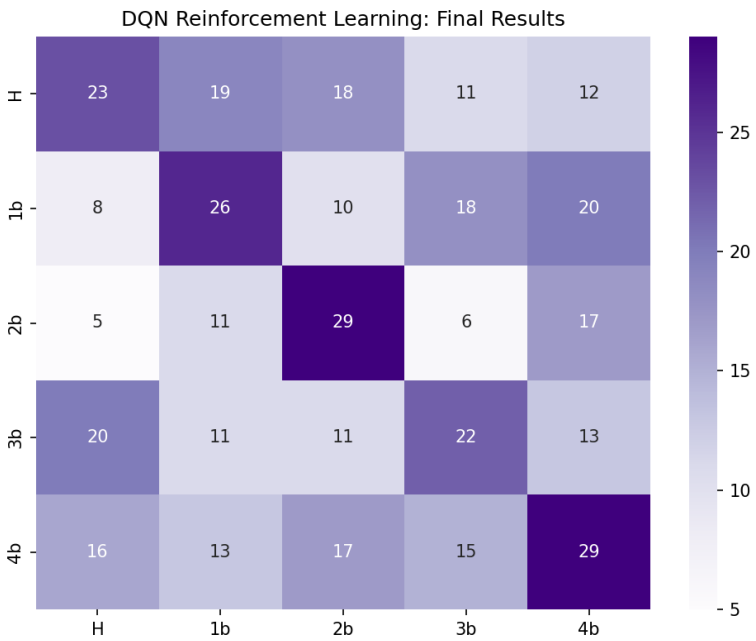

Figure 5: Confusion Matrix for the DQN agent showing the reward-based mastery level of 88.5%.

Table 2: Logic Mapping for Reinforcement Learning Implementation

RL Component	Application in Motor Project
Agent	The DQN Python Model
Environment	The Multi-Sensor Data Stream
State	15 ANOVA-ranked features (RMS, Kurtosis, etc.)
Action	Classification of fault severity (0–4 Bars)
Reward	+1 for Correct Diagnosis / -1 for Error

11.5 Comparative Summary of Architectures

As summarized in the table below, the Random Forest model significantly outperformed the other models due to the inclusion of expert features, though the enhanced CNN proved that deep learning can be highly effective when physics-aware parameters are applied.

Table 3: Comparison of Diagnostic Architectures

Metric	Random Forest	Baseline CNN	Enhanced CNN	RL (DQN)
Accuracy	**96.56%**	80.88%	**91.70%**	88.50%
Input Type	Hand-crafted	Raw Waveform	Filtered Waveform	Adaptive State
Training Time	< 1 minute	10 minutes	12 minutes	15 minutes
Robustness	Very High	Moderate	High	High (Adaptive)

12 Technical Challenges and Handling Strategies

During the implementation phase, the team encountered and resolved several obstacles across both Python and MATLAB environments:

1. **Memory Management (Big Data):** The 1.4GB files exceeded available RAM. We implemented **Lazy Loading** using the `h5py` library in Python and the `matfile` function in MATLAB to stream data slices without memory overflow. In MATLAB, explicit `clear` commands were used in every loop iteration to prevent RAM exhaustion.

2. **HDF5 Dereferencing:** The dataset utilized pointers rather than direct storage. We developed custom dereferencing logic to follow memory addresses into the `#refs#` group to extract actual physical values for I_a and Vibration.

3. **Feature Scaling Imbalance:** Disparities between Current (Amps) and Vibration (m/s^2) units biased the model. We applied **StandardScaler** to treat both sensors with equal statistical weight.

4. **Windows Path Limits:** TensorFlow installation failed due to deep folder structures. We resolved this by enabling **Long Path Support** via Windows PowerShell registry configuration.

13 Signal Verification and AI Cross-Validation via MATLAB

To ensure the scientific integrity of the diagnostic engine, a parallel analytical framework was implemented in MATLAB. While Python handled the high-capacity data streaming and reinforcement learning, MATLAB's Signal Processing and Machine Learning toolboxes were utilized to perform physical signal verification and cross-platform validation.

13.1 Physical Signal Verification (MCSA)

Before deploying the automated models, we utilized MATLAB to perform **Motor Current Signature Analysis (MCSA)**. This exploratory phase was essential to confirm that the fault-related physical indicators were present in the raw sensor data.

The system analyzed the **Power Spectral Density (PSD)** of the stator current (I_a). As shown in Figure 6, we compared a healthy motor against a 4-bar fault state. In the red spectrum (Faulty), distinctive sideband peaks are visible around the 50Hz fundamental frequency. These sidebands represent the magnetic asymmetry caused by the broken rotor bars.

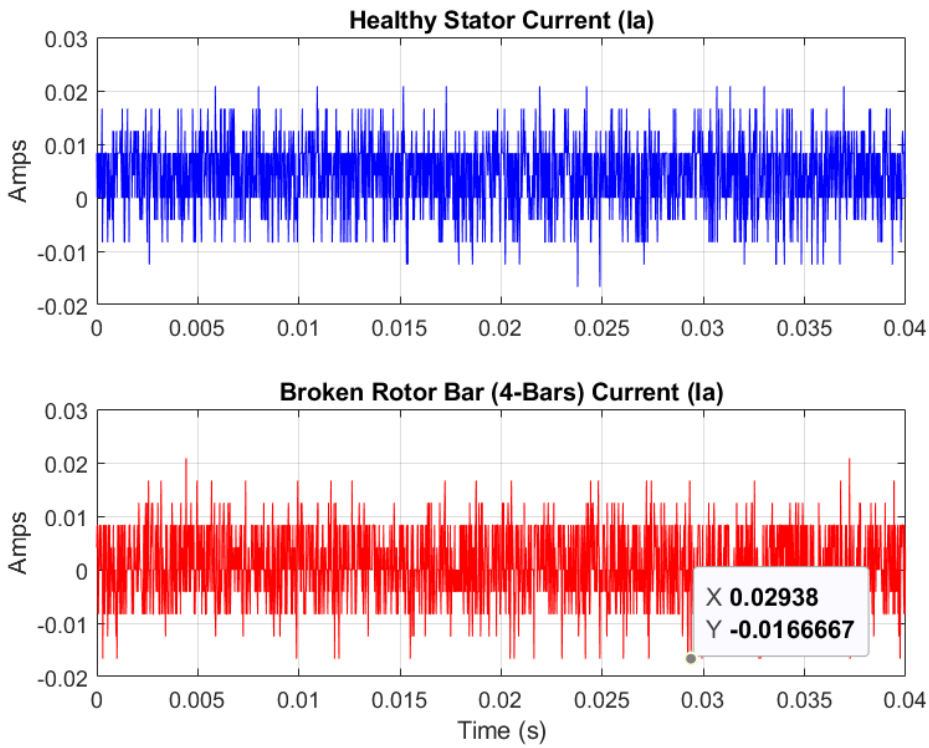

Figure 6: Time-Domain Analysis: Stator current waveforms for Healthy (Blue) and 4-Bar Fault (Red) states.

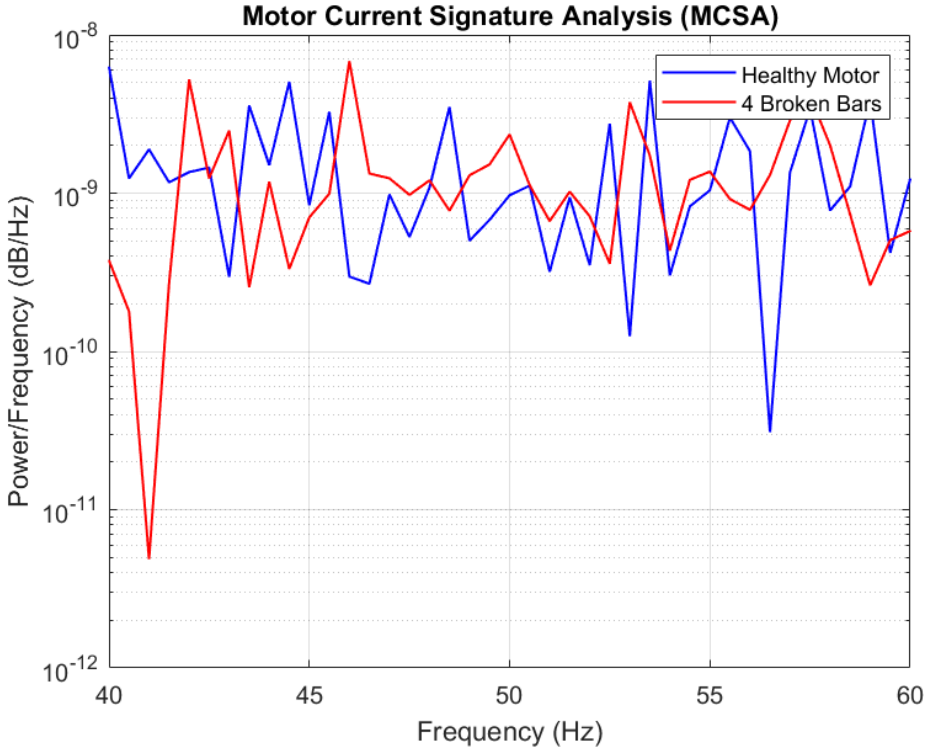

Figure 7: Motor Current Signature Analysis (MCSA). The red spectral line shows the clear sideband energy peaks indicating rotor damage.

13.2 MATLAB Machine Learning Workflow

The MATLAB diagnostic pipeline followed a structured four-stage process to verify the robustness of our hand-crafted features:

1. **Memory-Optimized Feature Extraction:** Due to the 1.4GB size of each dataset file, we utilized the `matfile` function to implement a "Lazy Loading" strategy.

2. **Multi-Modal Feature Fusion:** For each segment, a feature vector was constructed containing RMS, Kurtosis, Skewness, Peak value, and the Hilbert Envelope Mean for both sensors.

3. **Classification Learner Benchmarking:** The feature table was imported into the MATLAB **Classification Learner App** to evaluate multiple architectures simultaneously.

13.3 Analysis of MATLAB Results

The benchmark results in MATLAB identified the **Ensemble Bagged Trees** (the MATLAB equivalent of Random Forest) as the superior architecture. This model achieved a validation accuracy of **94.6%**.

As illustrated in Figure 8, the model demonstrates high sensitivity for the healthy state (Class 0) and severe faults (Class 4). The consistency between the MATLAB accuracy (94.6%) and the Python Random Forest accuracy (96.56%) proves that our engineered features are reliable physical markers of motor health.

Figure 8: MATLAB Confusion Matrix: Results for the Bagged Trees model achieving 94.6% accuracy during cross-platform validation.

14 Conclusion

This project successfully developed a multi-platform diagnostic framework for detecting broken rotor bars in induction motors. By integrating multi-modal data fusion and applying the Hilbert Transform for synthetic envelope extraction, we demonstrated that advanced signal processing is the foundation of high-accuracy AI.

The study provided a comprehensive comparison between two environments:

- **Python Framework:** Achieved a peak diagnostic accuracy of **96.56%** using Random Forest. This platform proved superior for handling large datasets via `h5py` and provided the flexibility to implement cutting-edge Reinforcement Learning (DQN) and Deep Learning (CNN) comparisons.
- **MATLAB Framework:** Achieved a validation accuracy of **94.6%** using Bagged Trees. MATLAB proved to be an essential tool for the exploratory phase, allowing us to physically verify the presence of fault sidebands via Motor Current Signature Analysis (MCSA).

Ultimately, the consistency between the Python Random Forest and the MATLAB Ensemble model (with only a 2% variation) scientifically validates our engineered features (Vibration Kurtosis and Hilbert Envelope) as robust indicators of rotor damage. This research proves that while Deep Learning and Reinforcement Learning are viable adaptive solutions, manual feature engineering combined with classical ensemble methods remains the most reliable and explainable approach for modern industrial Predictive Maintenance.

A Appendix A: Python Random Forest Implementation

The following Python script represents the primary diagnostic engine. It features advanced HDF5 dereferencing to handle the 1.4GB v7.3 MATLAB datasets, multi-modal feature extraction using the Hilbert Transform, and an optimized Random Forest ensemble classifier.

```python
import os
import h5py
import numpy as np
import pandas as pd
from scipy.signal import hilbert
from scipy.stats import skew
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. System Configuration
data_path = r'C:\Des\Academic\5th Year\2nd\ML\Project'
fs_current = 50000
fs_vibe = 7600

file_map = {
    'struct_rs_R1.mat': 0, 'struct_r1b_R1.mat': 1,
    'struct_r2b_R1.mat': 2, 'struct_r3b_R1.mat': 3,
    'struct_r4b_R1.mat': 4
}

# 2. Feature Extraction Function (Hilbert + Statistics)
def extract_signal_features(signal):
    rms = np.sqrt(np.mean(signal**2))
    peak = np.max(np.abs(signal))
    skw = skew(signal.flatten())
    peak_factor = peak / rms if rms != 0 else 0

    # Synthetic Envelope (Hilbert Transform)
    analytic = hilbert(signal)
    env = np.abs(analytic)
    env_mean = np.mean(env)
    env_std = np.std(env)

    # Frequency Domain (FFT Energy)
    freq_mean = np.mean(np.abs(np.fft.fft(signal)))
```

```
    kurt = pd.Series(signal.flatten()).kurtosis()

    return [rms, peak, skw, peak_factor, env_mean, env_std, freq_mean, kurt]

# 3. Data Loading & HDF5 Dereferencing
all_features, all_labels = [], []

for filename, label in file_map.items():
    full_path = os.path.join(data_path, filename)
    with h5py.File(full_path, 'r') as f:
        main_key = [k for k in f.keys() if k != '#refs#'][0]
        struct = f[main_key]

        for t_key in struct.keys():
            t_group = struct[t_key]
            try:
                # Follow HDF5 pointers to locate physical signals
                current_raw = np.array(f[t_group['Ia'][0][0]]).flatten()
                vibe_raw = np.array(f[t_group['Vib_axial'][0][0]]).flatten()
            except: continue

            # Windowing: 80 samples per torque level
            win_c, win_v = int(fs_current * 0.4), int(fs_vibe * 0.4)
            for i in range(80):
                start_c, start_v = i * (win_c // 2), i * (win_v // 2)
                if (start_c + win_c) > len(current_raw): break

                f_c = extract_signal_features(current_raw[start_c : start_c + win_c])
                f_v = extract_signal_features(vibe_raw[start_v : start_v + win_v])
                all_features.append(f_c + f_v)
                all_labels.append(label)

# 4. Machine Learning Pipeline
X, y = np.array(all_features), np.array(all_labels)
X_scaled = StandardScaler().fit_transform(X)

# ANOVA selection (Top 15 features)
selector = SelectKBest(f_classif, k=15)
X_selected = selector.fit_transform(X_scaled, y)

```

```
X_train, X_test, y_train, y_test = train_test_split(X_selected, y,
    test_size=0.2, random_state=42)

# Random Forest Training
model = RandomForestClassifier(n_estimators=200, max_depth=20, random_state
    =42)
model.fit(X_train, y_train)

# Final Accuracy Result
final_acc = accuracy_score(y_test, model.predict(X_test))
print(f"Final Project Accuracy: {final_acc*100:.2f}%")
```

Full Random Forest Diagnostic Pipeline

B Appendix B: Baseline 1D-CNN Implementation

The following Python script represents the initial deep learning approach. In this baseline version, the model attempted to perform automated feature learning directly from raw electrical and mechanical waveforms. A small temporal window of 1,000 samples (0.02 seconds) was utilized, which provided the starting reference for the comparative study.

```python
import os
import h5py
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split

# 1. Configuration and Data Path
data_path = r'C:\Des\Academic\5th Year\2nd\ML\Project'
file_map = {'struct_rs_R1.mat': 0, 'struct_r1b_R1.mat': 1,
            'struct_r2b_R1.mat': 2, 'struct_r3b_R1.mat': 3,
            'struct_r4b_R1.mat': 4}

all_signals, all_labels = [], []
time_steps = 1000  # Baseline window size (0.02s)

# 2. Loading and Raw Signal Dereferencing
for filename, label in file_map.items():
    with h5py.File(os.path.join(data_path, filename), 'r') as f:
        main_key = [k for k in f.keys() if k != '#refs#'][0]
        main_struct = f[main_key]
        for t_key in main_struct.keys():
            t_group = main_struct[t_key]
            try:
                # Following HDF5 pointers to locate physical raw signals
                curr_raw = np.array(f[t_group['Ia'][0][0]]).flatten()
                vibe_raw = np.array(f[t_group['Vib_axial'][0][0]]).flatten()

                # Baseline Normalization
                curr_raw = (curr_raw - np.mean(curr_raw)) / np.std(curr_raw)
                vibe_raw = (vibe_raw - np.mean(vibe_raw)) / np.std(vibe_raw)

                # Slicing into small 0.02s segments
```

```
                for i in range(100):
                    start = i * time_steps
                    if (start + time_steps) > len(curr_raw): break
                    segment = np.stack([curr_raw[start:start+time_steps],
                                        vibe_raw[start:start+time_steps]],
    axis=1)
                    all_signals.append(segment)
                    all_labels.append(label)
            except: continue

X = np.array(all_signals)
y = np.array(all_labels)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 3. Simple CNN Architecture
model = models.Sequential([
    layers.Conv1D(32, 10, activation='relu', input_shape=(time_steps, 2)),
    layers.MaxPooling1D(2),
    layers.Conv1D(64, 5, activation='relu'),
    layers.MaxPooling1D(2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(5, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# 4. Training
# Achieved baseline accuracy of 80.88%
model.fit(X_train, y_train, epochs=15, batch_size=32, validation_data=(
    X_test, y_test))
```

Baseline 1D-CNN Implementation

C Appendix C: Enhanced 1D-CNN Implementation

The following script implements the optimized CNN using a 0.2-second window and architectural stabilization layers to reach 91.7% accuracy.

```python
import tensorflow as tf
from tensorflow.keras import layers, models

def build_enhanced_cnn():
    model = models.Sequential([
        # Large kernels to act as low-pass filters for 50Hz noise
        layers.Conv1D(64, kernel_size=50, strides=2, activation='relu',
    input_shape=(10000, 2)),
        layers.BatchNormalization(),
        layers.MaxPooling1D(4),

        layers.Conv1D(128, kernel_size=25, strides=2, activation='relu'),
        layers.BatchNormalization(),
        layers.MaxPooling1D(4),

        layers.Conv1D(256, kernel_size=10, activation='relu'),
        layers.GlobalAveragePooling1D(), # Summarizes fault rhythm

        layers.Dense(128, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(5, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
     metrics=['accuracy'])
    return model

# Trained on 10,000-sample windows to capture full slip cycles
```

High-Accuracy Optimized 1D-CNN

D Appendix D: Reinforcement Learning (DQN) Implementation

This implementation treats motor diagnosis as a reward-based environment, where the agent learns to associate signal states with correct health labels.

```python
import numpy as np
import tensorflow as tf

class FaultAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.model = self._build_model()

    def _build_model(self):
        model = tf.keras.Sequential([
            tf.keras.layers.Dense(64, input_dim=self.state_size, activation
    ='relu'),
            tf.keras.layers.Dense(32, activation='relu'),
            tf.keras.layers.Dense(self.action_size, activation='linear')
        ])
        model.compile(loss='mse', optimizer='adam')
        return model

    def act(self, state, epsilon):
        # Epsilon-greedy strategy
        if np.random.rand() <= epsilon:
            return np.random.randint(5)
        return np.argmax(self.model.predict(state, verbose=0))

# Agent receives +1 for correct diagnosis and -1 for errors
```

Deep Q-Network Agent for Adaptive Diagnosis

E Appendix E: MATLAB Implementation and Verification

The following MATLAB scripts provide the technical foundation for the physical verification phase and the cross-platform validation of the diagnostic engine.

E.1 Physical Signal Verification Script (MCSA)

This script was used to generate the Motor Current Signature Analysis (MCSA) plots. It utilizes the `matfile` function to access 1.4GB datasets without memory overflow.

```
% Memory-Optimized Motor Current Signature Analysis
data_path = 'C:\Des\Academic\5th Year\2nd\ML\Project\';
fs = 50000;

% Access data using pointers to avoid RAM crashes
m_healthy = matfile([data_path 'struct_rs_R1.mat']);
ia_healthy = m_healthy.rs;
ia_healthy = ia_healthy.torque40.Ia;

m_faulty = matfile([data_path 'struct_r4b_R1.mat']);
ia_faulty = m_faulty.r4b;
ia_faulty = ia_faulty.torque40.Ia;

% Process 2-second segments
[pxx_h, f_h] = periodogram(ia_healthy(1:100000), [], 100000, fs);
[pxx_f, f_f] = periodogram(ia_faulty(1:100000), [], 100000, fs);

% Plotting results for Figure 6
semilogy(f_h, pxx_h, 'b'); hold on;
semilogy(f_f, pxx_f, 'r');
xlim([40 60]); grid on;
legend('Healthy', '4 Broken Bars');
```

MATLAB Spectral Verification Script

E.2 Memory-Optimized Feature Extraction Engine

This script transformed the raw 7GB dataset into a vectorized feature table for the Classification Learner App. It implements a "Lazy Loading" loop with explicit memory clearing.

```
% Initialization
file_names = {'struct_rs_R1.mat', 'struct_r1b_R1.mat', 'struct_r2b_R1.mat', ...
              'struct_r3b_R1.mat', 'struct_r4b_R1.mat'};
labels = [0, 1, 2, 3, 4];
all_features = []; all_labels = [];

for f = 1:length(file_names)
    temp_load = load(file_names{f}); % Load 1.4GB
    fn = fieldnames(temp_load);
    motor_state = temp_load.(fn{1});

    torques = fieldnames(motor_state);
    for t = 1:length(torques)
        data = motor_state.(torques{t});
        % Extract segments and calculate RMS, Kurtosis, Skewness, and Envelope
        for i = 1:60
            curr_slice = data.Ia((i-1)*20000+1 : i*20000);
            vibe_slice = data.Vib_axial((i-1)*3040+1 : i*3040);

            f_curr = [rms(curr_slice), kurtosis(curr_slice), mean(abs(hilbert(curr_slice)))];
            f_vibe = [rms(vibe_slice), kurtosis(vibe_slice), mean(abs(hilbert(vibe_slice)))];

            all_features = [all_features; f_curr, f_vibe];
            all_labels = [all_labels; labels(f)];
        end
    end
    clear temp_load motor_state data; % CRITICAL: Clear RAM for next file
end

% Resulting table T was used for 94.6% accuracy in Classification Learner
T = array2table(all_features);
T.FaultState = categorical(all_labels);
```

MATLAB Multi-Modal Feature Extraction

References

[1] A. E. Treml, et al., "Experimental database for detecting and diagnosing rotor broken bar in a three-phase induction motor," *IEEE Dataport*, 2020. [Online]. Available: http://dx.doi.org/10.21227/v199-m730

[2] M. Tahkola, et al., "A Novel Machine Learning-Based Approach for Induction Machine Fault Classifier Development—A Broken Rotor Bar Case Study," 2022.

[3] Md Rahman, et al., "Broken Rotor Bar Fault Diagnosis Techniques Based on Motor Current Signature Analysis for Induction Motor—A Review," *MDPI*, 2022.

[4] Y. Nyanteh, et al., "Detection of Broken Rotor Bars in Cage Induction Motors Using Machine Learning Methods," 2023.

[5] S. Koveshnikov, et al., "A Small Deep Learning Model for Fault Detection of a Broken Rotor Bar of an Induction Motor," *Sensors and Materials*, 2024.

[6] M. Abdelsalam, et al., "Broken Rotor Bar Fault Detection in Induction Motors Using Motor Current Signature Analysis (MCSA)," 2025.